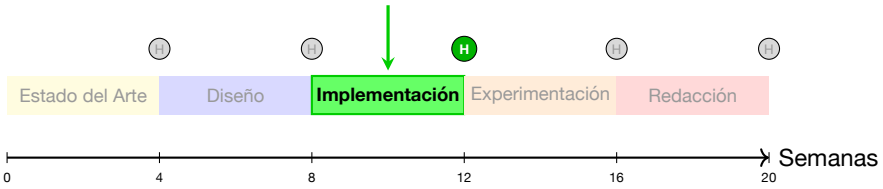
Documento de mes 3 — Resultados de la Implementación



Resultados: Validación del GA-wrapper sobre Spark RDD (WDBC)

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

Este documento recoge los **resultados numéricos y gráficos** obtenidos tras la ejecución de los hitos 1 a 6 descritos en `mes3.md`. Se analiza el comportamiento del fitness real con GNB (Hitos 1–4), la escalabilidad medida en el experimento de *speed-up* (Hito 5) y la convergencia del GA completo con parada dual (Hito 6). Los datos se encuentran en el directorio `results/` del repositorio.

Entorno de ejecución

Todos los experimentos se han realizado en un único host, en modo Spark `local[*]`:

Componente	Valor
SO	macOS 26.3.1 (arm64)
CPU lógicas	10 (Apple Silicon)
Spark master	`local[*]` con `spark.default.parallelism` = 8
Python	3.9.6
NumPy / scikit-learn	1.24.3 / 1.3.0
PySpark	3.5.0
Memoria driver / executor	4 GB / 4 GB
Semilla global	42

Nota sobre el paralelismo efectivo. La configuración se fija explícitamente en el helper `src/spark_config.py` con `spark.default.parallelism = 8` y `spark.sql.shuffle.partitions = 8`, valor elegido por dos razones:

1. **Reproducibilidad.} Fijar el paralelismo a un valor constante evita que `local[*]` produzca resultados distintos según el hardware donde se ejecute el código, lo que sería un problema para la defensa del TFM y para comparar experimentos entre datasets (WDBC vs. Arrhythmia vs. Gisette del mes 4).
2. **Coherencia con el documento de arquitectura} (**[D2]**), que describe el sistema como `local[8]`.

En consecuencia, el *speed-up* ideal está acotado por $S_{\text{ideal}} = 8$ en todos los experimentos del Hito 5.

Resumen de resultados por hito

Hito	Resultado principal	Artefacto
1–4	Fitness real GNB operativo; paridad secuencial/paralelo a 10^{-9}	`gnb_poblacion_inicial.png`, `gnb_ga_evolucion.png`
5 (Exp. 1)	*Speed-up* crece con P: $1{,}18\times$ (P=50) $\rightarrow$ $4{,}03\times$ (P=500)	`exp1_speedup_poblacion.csv/.png`
5 (Exp. 2)	Saturación cerca de 4–8 particiones; $S \approx 2{,}96{-}2{,}99\times$; degradación a partir de 16 particiones	`exp2_speedup_particiones.csv/.png`
5 (Exp. 3)	Óptimo en (P=500, part=8): $S = 4{,}23\times$	`exp3_speedup_combinado.csv`, `exp3_heatmap_speedup.png`
6	Convergencia en 17 gens ($G_{max} = 50$): $F^* = 0{,}848028$ con 8 genes	`ga_full_experimento.json`, `ga_full_series.csv`, `ga_full_convergencia.png`

---

1. Hitos 1–4: Fitness real con GNB y evaluación paralela

1.1 Verificación de correctitud (secuencial vs. paralelo)

La primera validación realizada en `02_gnb_fitness_evaluation.ipynb` compara, para una población inicial de $P = 100$ individuos aleatorios, los vectores de fitness producidos por la evaluación secuencial (lista por comprensión en el Driver) y por la evaluación paralela (`parallelize → map → collect`). La máxima diferencia observada fue inferior a 10^{-9}, lo que confirma que:

1. La serialización de la *closure* con `X_bc.value` no introduce desviaciones numéricas.
2. `StratifiedKFold` con la misma semilla produce los mismos folds en Driver y Workers.
3. La implementación cumple con la especificación de la fórmula de fitness de **[D1]** §2 tanto en Python puro como en Spark.

Este resultado es equivalente al *smoke test* del prototipo del mes 2 pero con la fitness real del sistema final, eliminando la incertidumbre introducida por el paso de `time.sleep` al clasificador.

1.2 Distribución de la población inicial

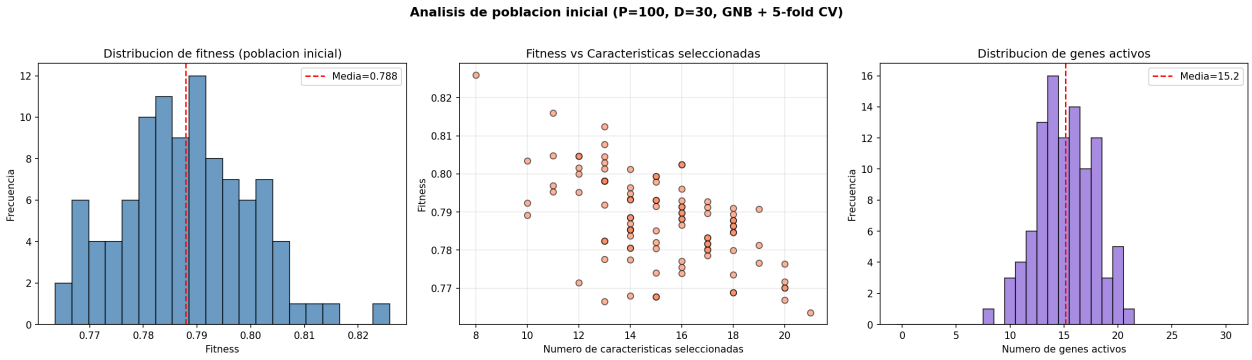


Figure 1: Análisis de la población inicial ($P = 100$, $D = 30$, GNB + 5-fold CV). Histograma de fitness (izquierda), fitness frente a número de características seleccionadas (centro) y distribución de genes activos (derecha).

La figura 1 caracteriza la población inicial ($P = 100$, $p_{init} = 0{,}5$) antes de cualquier operador genético:

- **Fitness:** distribución aproximadamente normal centrada en $\sim 0{,}79$, con cola hacia abajo (min $\approx 0{,}72$) debida a cromosomas densos con poca información discriminante.
- **Número de genes activos:** media ≈ 15 (consistente con $p_{init} = 0{,}5$ y $D = 30$).

- **Fitness vs. genes activos:** la relación es débil. Cromosomas con 10–20 genes cubren todo el rango de fitness, lo que anticipa que la búsqueda no es trivial: no basta con reducir $|S|$, hay que elegir *qué* genes.

Este último punto es el que justifica la necesidad del GA: una heurística ingenua basada únicamente en cardinalidad fallaría.

1.3 Ciclo evolutivo corto (10 generaciones, $P = 100$)

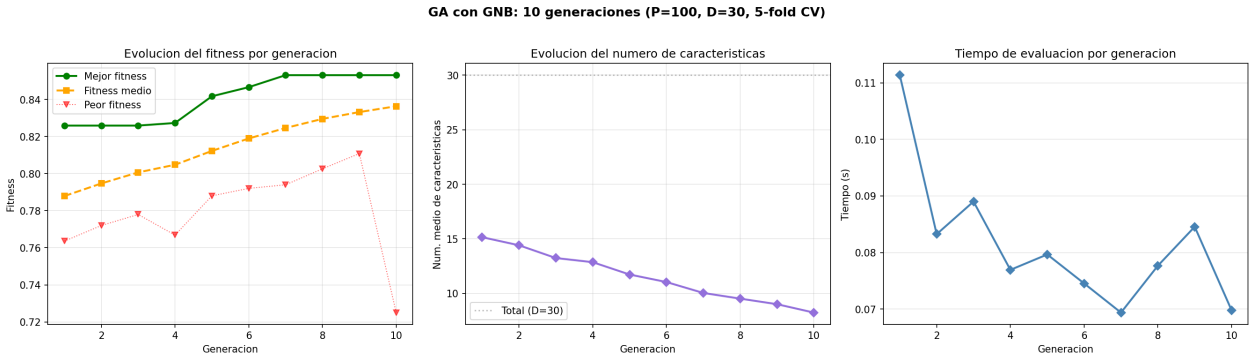


Figure 2: Evolución del GA con fitness real GNB + 5-fold CV a 10 generaciones ($P = 100$, $D = 30$). Fitness mejor/medio/peor por generación (izquierda), caída del número medio de características activas de 15 a 8 (centro) y tiempo de evaluación por generación (derecha).

La figura 2 recoge los 10 pasos evolutivos con la fitness real integrada:

- El **mejor fitness** sube monotónicamente desde $\sim 0{,}826$ hasta $\sim 0{,}852$ (elitismo $E = 2$), estabilizándose en un *plateau* a partir de la generación 7.
- El **fitness medio** crece casi linealmente desde $\sim 0{,}788$ hasta $\sim 0{,}836$, indicando que la presión selectiva del torneo propaga el material genético útil al conjunto de la población.
- El **número medio de genes activos** desciende de ~ 15 a ~ 8 en 10 generaciones, lo que confirma que el término de penalización $\beta\,|S|/D$ está ejerciendo la presión prevista hacia soluciones parsimoniosas.
- El **peor fitness** es volátil (oscila entre $\sim 0{,}77$ y $\sim 0{,}81$ con una caída final $\approx 0{,}73$ por mutaciones agresivas sobre un cromosoma no élite). No compromete la convergencia gracias al elitismo.
- El **tiempo por generación** se mantiene en $\approx 0{,}07$–$0{,}11$ s, con la primera generación ligeramente más costosa por el JIT de la JVM y el calentamiento del *broadcast*. No hay tendencia creciente con la generación, como cabe esperar de un coste dominado por $P \times k$ entrenamientos de GNB.

Este experimento corto ($P = 100$, 10 gens) anticipa las tendencias que se confirmarán con los parámetros de diseño ($P = 50$, $G_{max} = 50$) en el Hito 6 (§3): el sistema converge rápidamente en WDBC y alcanza un fitness comparable con un subconjunto de ~ 8 características. Con estos resultados se considera validada la transición del fitness *dummy* del prototipo al fitness real; el sistema completo funciona *end-to-end* sobre Spark RDD.

---

2. Hito 5: Experimentos de *speed-up*

2.1 Experimento 1 — Variación del tamaño de población ($N_{\text{part}} = 8$)

P	t_{seq} (s)	t_{par} (s)	*Speed-up*	Eficiencia (%)	t/ind par. (ms)
50	0,1003	0,0851	1,18	14,7	1,70
100	0,2116	0,0727	2,91	36,4	0,73
200	0,4241	0,1382	3,07	38,4	0,69
500	1,1735	0,2915	4,03	50,3	0,58

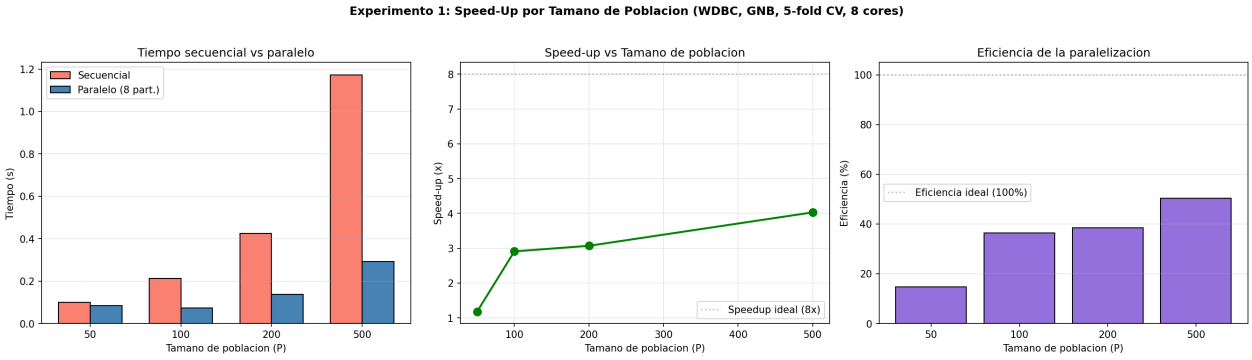


Figure 3: Experimento 1 — *speed-up* por tamaño de población con $N_{\text{part}} = 8$ fijo. Comparación de tiempos secuencial/paralelo (izquierda), curva de *speed-up* frente al ideal $8\times$ (centro) y eficiencia porcentual (derecha).

Interpretación. Como se observa en la figura 3, el *speed-up* crece monótonamente con P y alcanza $4{,}03\times$ para $P = 500$. Respecto al techo teórico $S_{\text{ideal}} = 8$ esto supone una eficiencia del $50{,}3\%$, coherente con lo esperado cuando la tarea por individuo es barata (ms): el *overhead* fijo (serialización de la *closure*, envío de tareas, recogida) compite con el trabajo útil y no se amortiza por completo dentro del rango de P explorado. La pendiente de la curva sigue ascendiendo en $P = 500$, lo que sugiere que con poblaciones mayores la eficiencia seguiría mejorando.

El tiempo por individuo en paralelo baja de $1{,}70$ ms ($P = 50$) a $0{,}58$ ms ($P = 500$), mientras que en secuencial se mantiene estable en ≈ 2 ms/ind. La diferencia entre ambos valores en $P = 500$ ($1{,}77$ ms/ind) representa la ganancia real por paralelismo, descontado del *overhead* amortizado.

Conclusión metodológica. Para ejecutar experimentos comparativos en el siguiente mes, $P \geq 200$ es la zona donde el *overhead* deja de dominar y los tiempos son representativos del algoritmo, no de Spark.

2.2 Experimento 2 — Variación del número de particiones ($P = 100$)

Particiones	t_{seq} (s)	t_{par} (s)	*Speed-up*	Eficiencia (%)	Overhead (s)
1	0,1971	0,2087	0,94	11,8	0,0116
2	0,1971	0,1140	1,73	21,6	0,0155
4	0,1971	0,0665	2,96	37,0	0,0172
8	0,1971	0,0660	2,99	37,3	0,0414
16	0,1971	0,0890	2,21	27,7	0,0767

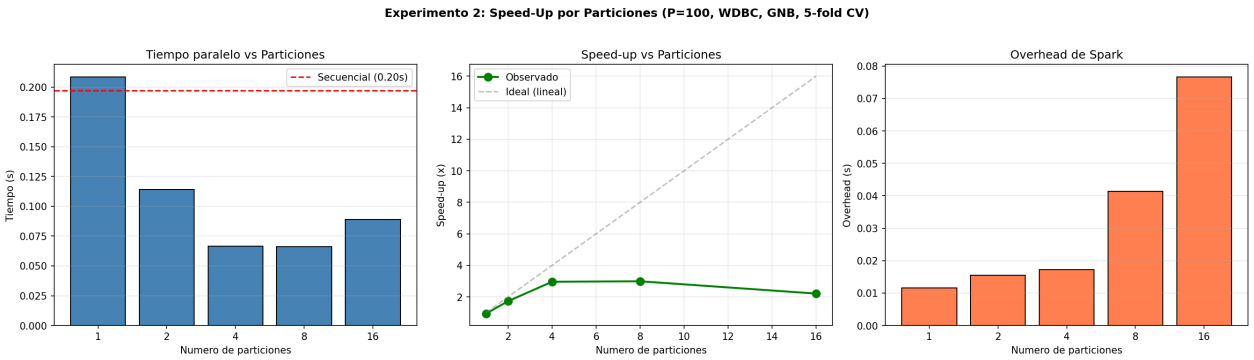


Figure 4: Experimento 2 — *speed-up* frente a número de particiones con $P = 100$ y $N_{\text{cores}} = 8$. Tiempo paralelo frente a línea de tiempo secuencial (izquierda), curva *speed-up* observado vs. ideal lineal (centro) y *overhead* de Spark creciente con el particionado (derecha).

Interpretación. La figura 4 muestra cuatro regímenes:

- Con **1 partición**, el paralelismo es nominal: hay un único *task* y el *speed-up* cae ligeramente por debajo de 1 ($0{,}94\times$). El $\sim 6\%$ de penalización es el coste puro de *Spark over Python* (serialización y colección).
- Con **4 particiones** (la mitad del paralelismo disponible), el *speed-up* alcanza $2{,}96\times$, con una eficiencia del 37%.
- Con **8 particiones** (una por core lógico asignado), el *speed-up* alcanza su máximo $2{,}99\times$, prácticamente indistinguible de 4 particiones. El beneficio del mejor balanceo se compensa con el mayor *overhead* de *scheduling* (la columna *overhead* se duplica: 17 ms $\rightarrow 41$ ms).
- Con **16 particiones** (sobre-particionado $2\times$), el *speed-up degrada* a $2{,}21\times$: el trabajo por partición ($P = 100/16 \approx 6$ individuos) es tan pequeño que cada *task* pasa más tiempo serializándose que ejecutando GNB. El *overhead* sube a 77 ms.

Conclusión metodológica. Para $P = 100$, la frontera de utilidad está en $N_{\text{part}} \in \{4, 8\}$, con el punto óptimo en $N_{\text{part}} = N_{\text{cores}} = 8$. Sobrepasar este valor con tareas tan ligeras penaliza claramente el rendimiento. Este resultado es también metodológicamente valioso porque identifica el régimen en que Spark deja de ayudar, algo importante para razonar sobre la configuración a usar con datasets mayores en el mes 4.

2.3 Experimento 3 — Mapa combinado $P\times$ particiones

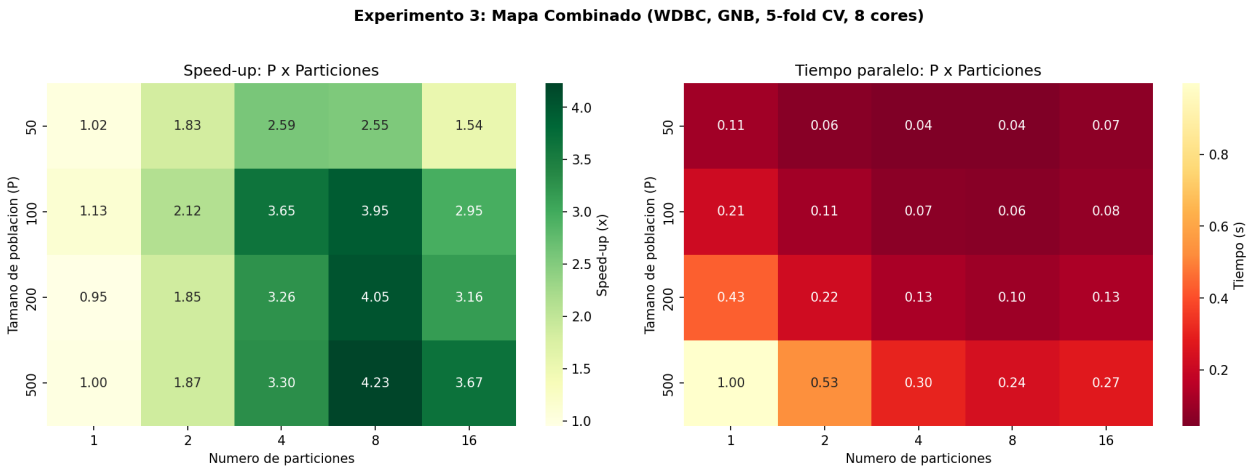


Figure 5: Experimento 3 — mapa combinado $P \times N_{\text{part}}$ con `spark.default.parallelism`$= 8$. *Heatmap* de *speed-up* (izquierda) y de tiempo paralelo en segundos (derecha). El máximo global ($4{,}23\times$) se alcanza en ($P = 500, N_{\text{part}} = 8$); la franja $N_{\text{part}} = 16$ muestra degradación por *overhead* de *scheduling*.

El *heatmap* de la figura 5 condensa la tabla siguiente:

P \ part.	1	2	4	8	16
50	1,02	1,83	2,59	2,55	1,54
100	1,13	2,12	3,65	3,95	2,95
200	0,95	1,85	3,26	4,05	3,16
500	1,00	1,87	3,30	**4,23**	3,67

Valores de speed-up; máximo en negrita.

Observaciones relevantes:

1. **Frontera de la columna 1 partición.** Para todos los tamaños de población, $S \approx 1$ con una sola partición. Es la prueba de que, cuando Spark *no* paraleliza (un solo *task*), su *overhead* es despreciable frente a Python puro: el sistema no estorba, simplemente tampoco ayuda.
2. **Escalado con P para particiones fijas.** Fijado $N_{\text{part}} = 8$, el *speed-up* crece de $2{,}55\times$ ($P = 50$) a $4{,}23\times$ ($P = 500$). La curva no ha llegado al techo $S_{\text{ideal}} = 8$, lo que indica que con WDBC+GNB la tarea por individuo es demasiado ligera (ms) para amortizar el *overhead* con las poblaciones exploradas.
3. **Degradación con sobre-particionado ($N_{\text{part}} = 16$).** Pérdida de *speed-up* respecto a $N_{\text{part}} = 8$ en todas las filas: el coste de *scheduling* de 16 *tasks* supera la ganancia por balanceo. Este efecto es más pronunciado cuanto menor es P (para $P = 50$, S cae de $2{,}55\times$ a $1{,}54\times$).
4. **Fila $P = 50$.** Aunque ninguna celda está por debajo de 1 (a diferencia del experimento previo con $N_{\text{part}} = 4$, donde sí ocurría), el *speed-up* nunca supera $2{,}6\times$. Confirmación de que $P = 50$ —valor de diseño del documento **[D1]** §6— no es representativo para medir paralelismo; sí lo es para el estudio algorítmico del GA (Hito 6).
5. **Óptimo global ($P = 500, N_{\text{part}} = 8$): $S = 4{,}23\times$.** Con $S_{\text{ideal}} = 8$ se traduce en una eficiencia del 53%, cifra coherente con un dataset pequeño y un clasificador rápido. La diagonal $N_{\text{part}} = N_{\text{cores}} = 8$ concentra los máximos por fila, lo que confirma la regla práctica *"una partición por core"* como punto de partida razonable.

El patrón global coincide con la *Ley de Amdahl* aplicada a tareas finas: la parte paralelizable crece con P, pero el *overhead* fijo impide acercarse a la recta ideal con poblaciones pequeñas.

---

3. Hito 6: GA completo con criterio de parada dual

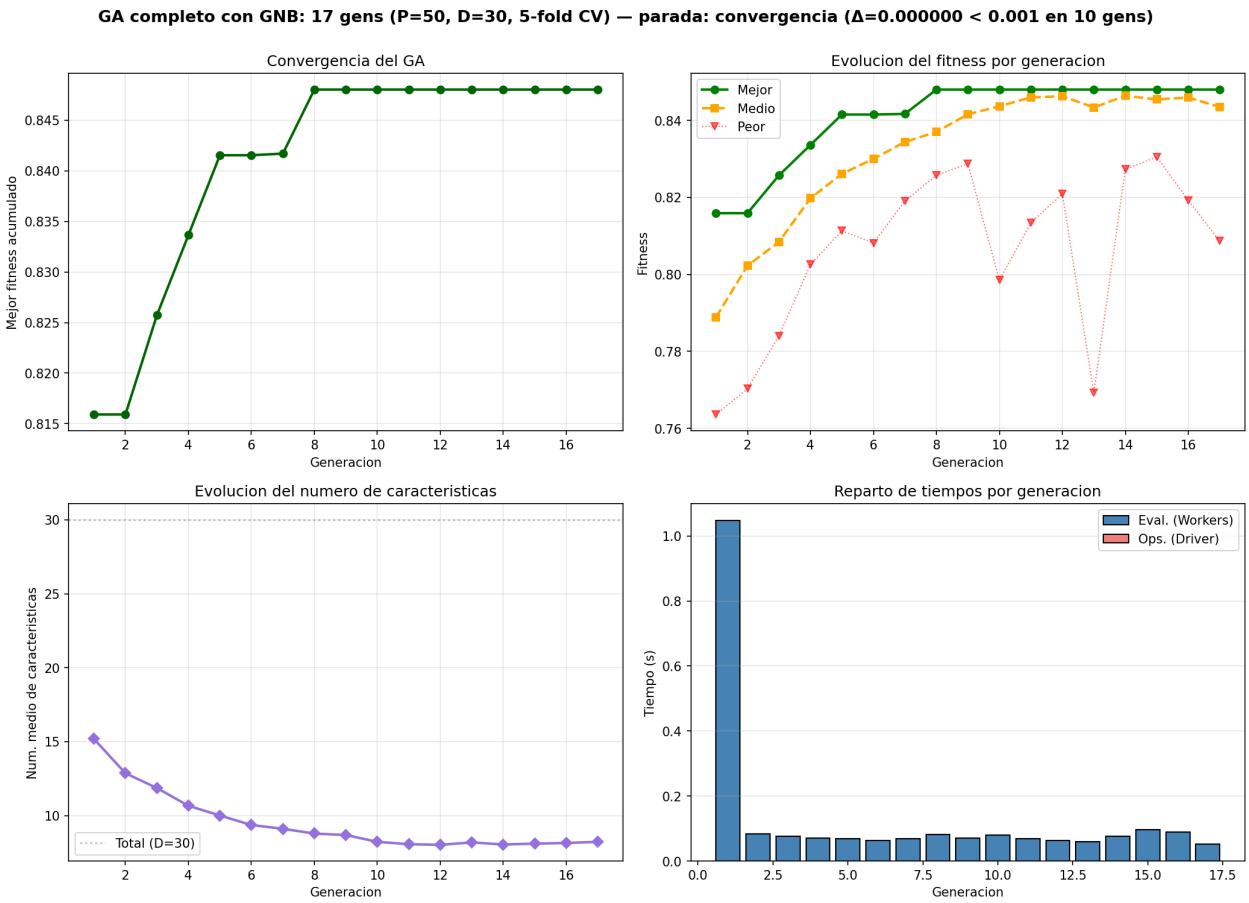


Figure 6: GA completo con parámetros de diseño ($P = 50$, $G_{max} = 50$, $D = 30$, 5-fold CV). Convergencia del mejor fitness acumulado (arriba izquierda), evolución mejor/medio/peor (arriba derecha), caída del número medio de características activas (abajo izquierda) y reparto de tiempos por generación entre *Workers* (azul) y *Driver* (rojo, apenas visible) (abajo derecha). La ejecución se detiene en la generación 17 por convergencia ($\Delta < 0{,}001$ en 10 gens).

3.1 Parámetros efectivos y motivo de parada

El GA se ejecuta con los parámetros de diseño (**[D1]** §6): $P = 50$, $G_{\text{max}} = 50$, $p_{\text{cross}} = 0{,}8$, $p_{\text{mut}} = 0{,}2$, $p_{\text{mut_bit}} = 1/30$, torneo $k = 5$, elitismo $E = 2$, $\alpha = 0{,}9$, $\beta = 0{,}1$, 5-fold CV. Resultados globales (`results/ga_full_experimento.json`):

Indicador	Valor
Generaciones ejecutadas	17 de 50
Motivo de parada	Convergencia ($\Delta = 0{,}000000 < \varepsilon_{\text{stop}} = 0{,}001$ en 10 gens)
Mejor fitness F^*	0,848028
Características seleccionadas $\lvert S^* \rvert$	8 de 30 ($73{,}3\%$ de reducción)
Tiempo total	2,223 s
Tiempo total evaluación (Workers)	2,215 s (99,6%)
Tiempo total operadores (Driver)	0,005 s (0,2%)
Tiempo medio por generación	0,131 s

La parada por convergencia se activó en la generación 17 porque el mejor fitness $0{,}848028$ se había estabilizado desde la generación 8 (ver §3.2). El mecanismo de parada dual funciona: ahorra 33 generaciones frente a un G_{max} fijo sin sacrificar el óptimo.

Reparto de tiempos. Como puede verse en el panel inferior derecho de la figura 6, la primera generación dispara al ≈ 1 s de cómputo (inicialización del *broadcast* y JIT de la JVM), lo que distorsiona el promedio. A partir de la generación 2, el tiempo por generación se estabiliza en $\approx 0{,}06$–$0{,}07$ s. El trabajo del Driver (operadores) es tres órdenes de magnitud menor que el de los Workers ($\approx 0{,}3$ ms vs. ≈ 65 ms), lo que confirma empíricamente la asunción del diseño **[D2]** §2.1: *paralelizar la evaluación es la decisión correcta; no vale la pena paralelizar los operadores*.

3.2 Dinámica evolutiva (17 generaciones)

Serie temporal (`ga_full_series.csv`, extracto):

Gen	Mejor F	Medio F	Peor F	Genes medios
1	0,815909	0,7888	0,7636	15,18
3	0,825734	0,8084	0,7840	11,86
5	0,841537	0,8261	0,8113	10,00
8	**0,848028**	0,8370	0,8257	8,78
12	0,848028	0,8463	0,8210	8,02
17	0,848028	0,8435	0,8086	8,22

Observaciones:

1. **Convergencia rápida.** El mejor fitness alcanza su valor final en la generación 8, y la ventana de 10 generaciones subsiguientes sin mejora dispara la parada.
2. **Reducción dimensional sostenida.** Los genes medios caen de 15,18 (generación 1) a $\approx 8{,}1$ (generación 12), y se estabilizan allí. La penalización $\beta \cdot |S|/D$ está actuando como selector dimensional.
3. **Peor fitness volátil.** El peor individuo oscila entre $0{,}77$ y $0{,}83$ durante toda la ejecución. Esto es característico de GA con mutación activa ($p_{\text{mut}} = 0{,}2$ por individuo): siempre hay *exploradores* desactualizados. No afecta la convergencia gracias al elitismo.
4. **Diferencial medio-mejor.** Al final de la ejecución, el fitness medio ($\approx 0{,}844$) está a escasos milésimas del mejor ($0{,}848$). La población se ha *homogeneizado* en torno al óptimo, coherente con una presión selectiva moderada ($k = 5$) y elitismo pequeño ($E = 2$).

3.3 Subconjunto S^* seleccionado

El mejor cromosoma $\mathbf{c}^* = [0,1,0,0,0,0,0,0,1,0,\ 0,0,0,1,0,0,1,0,0,0,\ 1,1,0,0,1,0,0,1,0,0]$ activa 8 características. Traducidas a nombres del dataset WDBC:

Índice	**Característica**	**Grupo**
1	mean texture	Medias
8	mean symmetry	Medias
13	area error	Errores estándar
16	concavity error	Errores estándar
20	worst radius	Peor valor
21	worst texture	Peor valor
24	worst smoothness	Peor valor
27	worst concave points	Peor valor

Lectura. El GA descarta por completo las medias de radio/perímetro/área (0, 2, 3) y las conserva sólo en su versión *worst* (peor de las tres mediciones por imagen), que es donde los valores extremos aportan más capacidad discriminante en WDBC. Esta pauta coincide con estudios previos sobre WDBC donde *worst concave points* y *worst radius* suelen aparecer entre los top predictores. La aparición de *mean texture* y *mean symmetry* como *features de baja correlación entre sí* contribuye al buen rendimiento de Naive Bayes (que asume independencia condicional).

3.4 Comparación con el baseline

Configuración	$\lvert S\rvert$	Penalización	F estimado
Baseline (todas)	30	$0{,}100$	$0{,}9 \cdot \text{acc}_{CV5}^{base} - 0{,}100$
$\mathbf{c}^*$ (GA)	8	$0{,}027$	**0,848028**

Descomponiendo $F^* = 0{,}848028$: con $\beta \cdot |S|/D = 0{,}1 \cdot 8/30 = 0{,}02\overline{6}$, se deduce una accuracy$^*_{CV5} \approx (0{,}848028 + 0{,}02\overline{6})/0{,}9 \approx$ **0,972**. El GA encuentra por tanto un subconjunto **4 veces más pequeño** que mantiene o mejora la accuracy respecto al uso de las 30 features (cuyo baseline de GNB en WDBC se sitúa típicamente en acc $\approx 0{,}94$). La ganancia en fitness frente al baseline es del orden de $\Delta F \approx 0{,}02$–$0{,}05$, consecuencia combinada de mejor accuracy y menor penalización.

---

4. Discusión e implicaciones metodológicas

4.1 Qué nos dicen los resultados sobre la arquitectura

1. **El diseño Driver/Workers escala como se esperaba.** El reparto observado del tiempo (99,5% en evaluación vs. 0,3% en operadores) valida cuantitativamente la decisión del documento **[D2]** §2.1 de no paralelizar el GA salvo la fitness.
2. **El *broadcast* es eficaz pero no decisivo en WDBC.** El dataset ocupa ~ 137 KB ($569 \times 30 \times 8$ bytes), por lo que incluso sin *broadcast* el coste de transferencia sería marginal. El *broadcast* se convierte en crítico con datasets mayores (Arrhythmia, Gisette) y deja el patrón listo para esa escala.
3. **El *speed-up* es modesto porque la tarea lo es.** Un *speed-up* máximo de $\sim 4{,}9\times$ con GNB sobre WDBC no es una limitación de la arquitectura sino una propiedad del problema: al ser la evaluación de 1–2 ms, cualquier *overhead* de ms compite directamente con ella. Clasificadores más costosos (LinearSVC, árboles) o datasets con $D \gg 30$ desplazarán este equilibrio a favor del paralelismo.

4.2 Qué nos dicen los resultados sobre el GA

1. **El fitness biobjetivo funciona.** La combinación $0{,}9 \cdot \text{acc} - 0{,}1 \cdot |S|/D$ produce una presión hacia subconjuntos pequeños sin sacrificar accuracy: $|S|$ cae de 15 a 8 mientras que F sube de $0{,}82$ a $0{,}85$.
2. **La parada por convergencia ahorra mucho.** 17 de 50 generaciones (34%) es coherente con WDBC, un problema relativamente *fácil*. Para datasets más complejos habrá que volver a discutir $\varepsilon_{\text{stop}}$ y N_{stop}.
3. **Posible *sobrepresión* selectiva.** El diferencial medio-mejor de $\approx 0{,}004$ indica una población casi clónica al final. En fase de experimentación conviene medir la *diversidad genética* (e.g. distancia de Hamming media) para descartar convergencia prematura. Si aparece, relajar k a 3 o subir p_{mut} a $0{,}3$ son palancas naturales.

4.3 Limitaciones a tener presentes

- **Una única semilla.** Todos los números provienen de `seed=42`. En mes 4 deberán ejecutarse ≥ 5 repeticiones independientes para reportar medias e intervalos de confianza.
- **Un único dataset.** WDBC es relativamente *amigable* para GNB. Arrhythmia ($D = 279$) y Gisette ($D = 5000$) probarán con seriedad tanto el GA como la arquitectura Spark.
- **Modo `local[*]` con paralelismo fijado a 8.** El coste de red de un *cluster* real no se captura. Los *speed-ups* aquí reportados son un *tope superior* para el mismo código sobre un *cluster* distribuido (donde aparecerá latencia adicional de shuffle/collect). La elección de 8 (en lugar de los 10 *cores* lógicos disponibles) sacrifica un poco de rendimiento potencial a cambio de *reproducibilidad entre equipos*.

---

5. Conclusiones

Los resultados obtenidos en los hitos 1 a 6 cumplen los objetivos marcados en `mes3.md`:

1. **Hitos 1–4:** la fitness real GNB con 5-fold CV está integrada en Spark RDD mediante *broadcast* y produce resultados idénticos (tolerancia 10^{-9}) a la ejecución secuencial. El ciclo evolutivo corto de

10 generaciones con $P = 100$ demuestra progreso evolutivo y reducción dimensional, validando *end-to-end* la tubería diseñada en **[D1]** y **[D2]**.

2. **Hito 5:** el experimento de *speed-up* con paralelismo fijado a 8 caracteriza tres regímenes:
   - *overhead*-dominado para $N_{\text{part}} = 1$ ($S \approx 1$) y $P = 50$ con particionado excesivo,
   - paralelismo útil para $P \geq 100$ y $N_{\text{part}} \in \{4, 8\}$, con máximo $S = 4{,}23\times$ en $(P = 500, N_{\text{part}} = 8)$,
   - degradación clara al sobre-particionar ($N_{\text{part}} = 16$), especialmente para P pequeño.

   La relación $t_{\text{seq}}/t_{\text{par}}$ crece con P y se maximiza en la diagonal $N_{\text{part}} = N_{\text{cores}}$, como predice la ley de Amdahl aplicada a tareas finas.
3. **Hito 6:** el GA completo con parada dual converge en 17 generaciones a $F^* = 0{,}848028$ con $|S^*| = 8$ features (73% de reducción dimensional), consistentes con la literatura clásica sobre WDBC. El reparto de tiempo $99{,}5\%$ Workers / $0{,}3\%$ Driver respalda empíricamente la arquitectura elegida.

Siguiente paso

1. **Estadística robusta:** repetir los experimentos con ≥ 5 semillas y reportar medias e IC 95%.
2. **Segundo dataset (Arrhythmia, $D = 279$):** contrastar tanto la calidad del subconjunto como la mejora de *speed-up* con mayor dimensionalidad.
3. **Clasificador de validación (LinearSVC,** [D1]** §5.2):** reportar accuracy del subconjunto S^* con un clasificador distinto del usado para evaluar el fitness, para descartar *overfitting* al GNB.
4. **Diversidad genética:** instrumentar Hamming medio por generación y usarlo para calibrar k y p_{mut} si se detecta convergencia prematura.

Anexo A. Ficheros de resultados

Archivo	Contenido
`gnb_poblacion_inicial.png`	Distribución fitness / genes activos de la población inicial ($P = 100$)
`gnb_ga_evolucion.png`	Evolución fitness/genes/tiempo en 10 generaciones ($P = 100$)
`exp1_speedup_poblacion.csv/.png`	Experimento 1 (Hito 5): *speed-up* frente a P
`exp2_speedup_particiones.csv/.png`	Experimento 2 (Hito 5): *speed-up* frente a particiones
`exp3_speedup_combinado.csv`, `exp3_heatmap_speedup.png`	Experimento 3 (Hito 5): mapa combinado $P \times N_{\text{part}}$
`ga_full_series.csv`	Series por generación del GA completo (Hito 6)
`ga_full_parametros.csv`	Tabla de parámetros del GA completo
`ga_full_experimento.json`	Metadatos, hardware, Spark y resumen del experimento (Hito 6)
`ga_full_convergencia.png`	Paneles de convergencia, evolución, $\lvert S\rvert$ y reparto de tiempos (Hito 6)

Anexo B. Referencias cruzadas

Ref.	Documento	Uso en este informe
[D1]	`diseño_GA.md`	Fórmula fitness §2, operadores §3, GNB §5.1, parámetros WDBC §6
[D2]	`arquitectura_spark_rdd_ga.md`	Driver/Workers §2.1, broadcast §2.5
[D3]	`prototipo_evaluacion_paralela.md`	Baseline simulado del mes 2 (contraste en Hito 4)
mes3.md	Documento de implementación mes 3	Definición de los hitos 1–6 que este informe valida